

Cloud Computing and Virtualization

Bachelors in Computer Engineering

**Prepared by Er. Ganga Gautam,
Assistant Professor, Pokhara University**

Chapter 5

Security in cloud Computing(5hr)

Outlines:

1. Cloud Security
 - 1.1. Cloud Security Threats
2. Security as a Service
3. Dimension of Cloud Security
4. Cloud Security Mechanism
 - 4.1. SSO, IAM, Hashing and Digital Signatures
5. Disaster Management in cloud computing
6. Challenges of Cloud Security

cloud security

- A set of policies, technologies, controls, and practices designed to protect:
 - a. Data
 - b. Applications
 - c. Services
 - d. Infrastructure in cloud environments.
- Ensures confidentiality, integrity, and availability (CIA) of digital assets.
- Covers hardware, software, storage, networks, and virtualization layers
- Implemented jointly by cloud providers & consumers.

Why Cloud Security Matters

- Rapid cloud adoption → Expanded attack surface.
- Sensitive data is hosted on shared infrastructures.
- Increasing incidents of:
 - Data breaches
 - Account hijacking
 - Insecure APIs
 - Misconfigurations
- Compliance needs (GDPR, HIPAA, ISO 27001) demand strong security.
- Example: Public S3 bucket exposure leads to massive data leaks.

Cloud Security Goals

- **Confidentiality**
 - Only authorized users access data.
- **Integrity**
 - Data remains accurate and unaltered.
- **Availability**
 - Resources are accessible when needed.
- **Accountability**
 - Every action can be traced to an entity.
- **Compliance**
 - Meeting standards like GDPR, ISO 27001.

Traditional vs. Cloud Security

Shared Responsibility Model

- Core concept of cloud security.
- **Provider:**
 - Responsible for security of the cloud.
 - Physical data centers
 - Network & hypervisor layer
 - Hardware security
- **Customer:**
 - Responsible for security in the cloud.
 - Data classification & protection
 - Identity and access management (IAM)
 - Application-level security
- Responsibility varies by model (IaaS, PaaS, SaaS).
- **Examples:**
 - AWS: Physical, network, hypervisor → AWS responsibility.
 - User: IAM, data encryption, access control → User responsibility.

Cloud Security Principles (CIA Triad)

1. Confidentiality:

- Protect data from unauthorized access.
- Encryption, authentication, access control.

2. Integrity:

- Ensure data is accurate and untampered.
- Hashing, digital signatures.

3. Availability:

- Systems accessible when needed.
- Redundancy, backup, fault tolerance.



Cloud Security Lifecycle

1. **Assess:** Identify assets & risks.
2. **Protect:** Apply controls (IAM, encryption).
3. **Detect:** Continuous monitoring.
4. **Respond:** Incident response & logging.
5. **Recover:** Backup & restoration.

Case Study: AWS S3 Data Breach

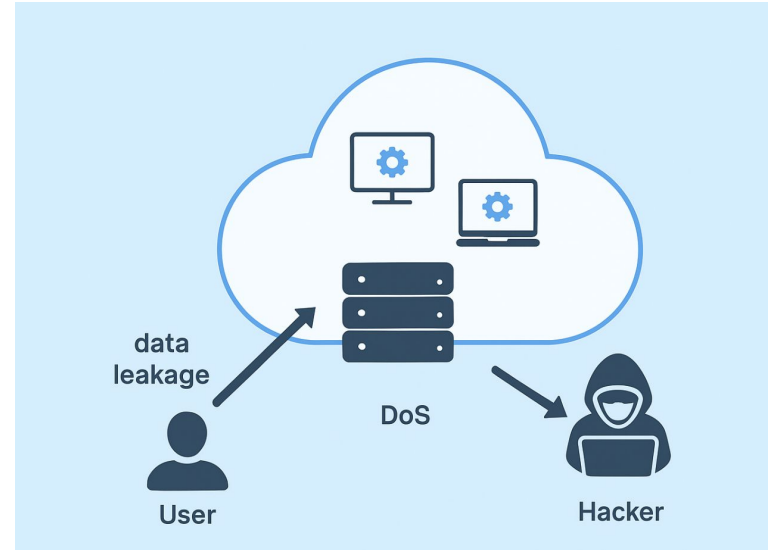
Scenario:

- Public S3 bucket exposed 540 million records.
- Cause: Misconfigured permissions.
- Prevention: IAM policies + Access Analyzer.

It's not about encryption only — it's about configuration hygiene.

Cloud Security Threats

- Cloud threats differ from traditional IT because of:
 - Multi-tenancy
 - Data outsourcing
 - Virtualization layers
 - Dynamic resource allocation
- Example:
 - In AWS, multiple tenants share the same physical server— increasing exposure to side-channel attacks.



Common Cloud Security Threats

According to CSA(Cloud Security Alliance) “Egregious 11” (latest report):

1. Data Breaches
2. Misconfiguration
3. Insecure Interfaces/APIs
4. Account Hijacking
5. Insider Threats
6. Weak Access Management
7. Inadequate Change Control
8. Lack of Visibility
9. Denial of Service
10. Abuse of Cloud Resources
11. Shared Technology Vulnerabilities

1. Data Breaches

- Unauthorized access, exposure, or theft of sensitive data stored or processed in the cloud.
- **Causes:**
 - Weak encryption
 - Poor access control or stolen credentials
 - Misconfigured cloud storage (e.g., open S3 buckets)
- **Impact:**
 - Loss of confidential information
 - Legal penalties, reputational damage
- **Mitigation:**
 - Encrypt data (in transit & at rest), enable strong IAM, and monitor access logs.

2. Misconfiguration

- Incorrect setup of cloud resources that exposes them to vulnerabilities or unauthorized access.
- **Examples:**
 - Publicly open storage buckets
 - Unrestricted firewall rules
 - Disabled monitoring/logging
- **Impact:**
 - Leads to accidental data exposure or full compromise of cloud assets.
- **Mitigation:**
 - Automated configuration audits, compliance tools (AWS Config, Azure Policy).

3. Insecure Interfaces and APIs

- Cloud services rely on APIs; insecure APIs can be exploited to gain unauthorized access or control.
- **Causes:**
 - Weak authentication
 - Lack of encryption or input validation
 - Exposed endpoints
- **Impact:**
 - Attackers may manipulate APIs to steal data or hijack resources.
- **Mitigation:**
 - Use secure tokens (OAuth 2.0), API gateways, input validation, and strong authentication.

4. Account Hijacking

- Attackers steal cloud user credentials to gain unauthorized access to resources.
- **Methods:**
 - Phishing, keylogging, or credential reuse
 - Exploiting weak passwords or tokens
- **Impact:**
 - Complete compromise of virtual machines, databases, or storage.
- **Mitigation:**
 - Use MFA, password rotation, behavior monitoring, and secure session handling.

5. Insider Threats

- Malicious, careless, or compromised employees or contractors who misuse legitimate access.
- **Types:**
 - **Malicious insider:**
 - Intentionally leaks data
 - **Negligent insider:**
 - Misconfigures systems
 - **Compromised insider:**
 - Account hacked
- **Impact:**
 - Hard to detect and can cause massive data loss.
- **Mitigation:**
 - Behavior analytics (UEBA), least privilege, audit trails, and segregation of duties.

6. Weak Access Management

- Inadequate enforcement of identity and access policies, leading to unauthorized access.
- **Causes:**
 - Over-privileged accounts
 - Lack of MFA or role-based control
 - Shared credentials
- **Impact:**
 - Privilege escalation and unauthorized data modification.
- **Mitigation:**
 - Strong IAM policies, least privilege principle, MFA, key rotation.

7. Inadequate Change Control

- Failure to properly track, review, or approve configuration and code changes in the cloud.
- **Examples:**
 - Direct manual edits without testing
 - Lack of rollback or versioning controls
- **Impact:**
 - Introduces security gaps and system instability.
- **Mitigation:**
 - Follow DevSecOps with version control (Git), change approval workflows, and automated deployment pipelines.

8. Lack of Visibility

- Limited insight into who is accessing what and how cloud resources are used.
- **Causes:**
 - Multi-cloud complexity
 - Inconsistent or disabled logging
- **Impact:**
 - Delayed detection of attacks and compliance violations.
- **Mitigation:**
 - Centralized monitoring (CloudTrail, Azure Monitor), SIEM integration, and CASB tools.

9. Denial of Service (DoS)

- Attackers overwhelm cloud resources, making applications or services unavailable.
- **Types:**
 - **Volumetric attacks:**
 - High traffic floods
 - **Protocol attacks:**
 - Exploit network resources
 - **Application-layer attacks:**
 - Target web apps
- **Impact:**
 - Service downtime, loss of revenue, poor customer trust.
- **Mitigation:**
 - Use autoscaling, CDN protection, DDoS mitigation tools (AWS Shield, Azure DDoS Protection).

10. Abuse of Cloud Resources

- Attackers exploit cloud infrastructure to perform malicious activities.
- **Examples:**
 - Cryptomining
 - Hosting malware or phishing sites
 - Launching further cyberattacks
- **Impact:**
 - Increased costs, IP blacklisting, reputational harm.
- **Mitigation:**
 - Usage monitoring, anomaly detection, strict resource limits, and user verification.

11. Shared Technology Vulnerabilities

- Flaws in the shared components (e.g., hypervisor, containers, network fabric) used by multiple tenants in a cloud.
- **Causes:**
 - Multi-tenancy without strong isolation
 - Exploitable hypervisor or virtualization layer
- **Impact:**
 - Attackers could escape from one virtual machine to another (VM escape).
- **Mitigation:**
 - Regular patching, strong isolation, hardware-assisted virtualization security.

Security as a Service

- SECaaS i.e, “Security delivered via cloud-based subscription model.”
- Functions are hosted by third-party providers.
- Examples:
 - Antivirus, identity management, email filtering, DLP.
- Analogous to SaaS, but focused on protection instead of productivity.

Why SECaaS Matters

- **Scalable Protection:**
 - Instantly adjusts to traffic or user growth.
- **Reduced Costs:**
 - No on-premise hardware or updates needed.
- **Real-time Updates:**
 - Continuous patching and monitoring.
- **Anywhere Access:**
 - Works across distributed teams and hybrid clouds.

Key Categories of SECaaS

1. **Identity & Access Management (IAM):**
 - Controls who can access what.
2. **Email Security:**
 - Filters spam, phishing, and malicious attachments.
3. **Web Security:**
 - Protects browsing and web apps from malware or XSS.
4. **Data Loss Prevention (DLP):**
 - Monitors and prevents unauthorized data sharing.

5. **Intrusion Detection & Prevention (IDS/IPS):**
 - Detect and stop threats in real time.
6. **Encryption & Key Management:**
 - Protects sensitive data at rest and in transit.
7. **Security Information & Event Management (SIEM):**
 - Centralizes logs for real-time threat analytics.

SECaaS Providers (Examples)

Provider	Service Focus	Example Features
Zscaler	Web & Zero Trust Security	Secure access to SaaS apps
Cisco Umbrella	DNS & Cloud Firewall	Threat intelligence
Palo Alto Prisma Cloud	Cloud-native Security	Workload protection
AWS GuardDuty	Threat Detection	ML-based anomaly detection

These providers offer specialized SECaaS models. AWS GuardDuty, for example, integrates with IAM roles to monitor suspicious behavior automatically.

Advantages of SECaaS

- Cost-efficient subscription model
- Scalable with cloud usage
- Always updated (patches, threat feeds)
- Centralized control panel for all users
- Easy integration with cloud APIs (AWS, Azure, GCP)

Dimensions of Cloud Security

Cloud Security consists of eight dimensions that collectively protect data, identities, networks, applications, infrastructure, compliance, monitoring, and responsibility sharing in cloud environments.

1. Data Security
2. Identity and Access Management (IAM)
3. Network Security
4. Application Security
5. Infrastructure Security
6. Compliance and Legal Security
7. Monitoring and Incident Response
8. Shared Responsibility Model

Data Security in the Cloud

- Prevent unauthorized access or modification.
- Techniques:
 - **Encryption:**
 - Data at Rest: Encrypt data stored in databases (AWS KMS, Azure Key Vault).
 - Data in Transit: Use TLS/SSL encryption for secure transmission.
 - **Data Access Controls:**
 - Implement Role-Based Access Control (RBAC).
 - Apply principle of least privilege (PoLP).
 - **Backup & Disaster Recovery:**
 - Use cloud snapshot backups (AWS Backup, Azure Backup).
 - Enable geo-redundant storage for disaster recovery.
 - **Firewalls and intrusion detection systems**
 - **Multi-Factor Authentication (MFA)**

Identity and Access Management (IAM)

- Ensure only authorized users and services can access cloud resources.
- **Techniques:**
 - **Authentication**
 - Passwords, certificates, and biometric authentication
 - **Authorization**
 - IAM roles and policies to define permissions
 - **Access Management**
 - Single Sign-On (SSO) integration
 - MFA for critical operations
 - **Examples**
 - AWS IAM, Azure Active Directory (Entra ID), Google Cloud IAM

Network Security in Cloud

- Protect cloud networks from unauthorized access and cyber-attacks.
- **Firewall & Security Groups**
 - Control traffic at network level.
- **Virtual Private Cloud (VPC)**
 - Isolated cloud network for security.
- **Zero Trust Model**
 - No device/user is trusted by default.
- **DDoS Protection**
 - Use AWS Shield, Azure DDoS Protection, Cloudflare.
- **IDS/IPS**
 - real-time detection.
- **Network segmentation and VPNs for isolation.**

Application Security

- Protect cloud applications from vulnerabilities and attacks.

Techniques:

- **Secure Software Development**
 - Follow secure coding standards
- **Vulnerability Management**
 - Regular scanning and patching
- **Web Application Firewalls (WAF)**
 - Protect against SQL injection, XSS attacks
- **Runtime Protection**
 - Application monitoring and logging
- **Examples**
 - AWS WAF, Azure Application Gateway, Google Cloud Armor

Infrastructure Security

- Secure physical and virtual cloud infrastructure.

Techniques:

- **Physical Security**
 - Secured data centers with surveillance and access control
- **Virtualization Security**
 - Secure hypervisors and virtual machines
- **Configuration Management**
 - Harden OS and cloud resource configurations
- **Patch Management**
 - Regular updates of servers and firmware
- **Responsibility**
 - Mostly handled by cloud service providers

Compliance & Legal Security

- Ensure cloud usage follows laws, regulations, and industry standards.

Techniques:

- **Regulatory Compliance**
 - GDPR, HIPAA, ISO 27001, SOC 2
- **Auditing & Reporting**
 - Continuous compliance monitoring
- **Data Residency Controls**
 - Store data in approved geographic
- **Policy Enforcement**
 - Cloud governance and risk manage

Compliance Standard	Industry	Description
GDPR	General	Protects user data privacy (Europe).
HIPAA	Healthcare	Protects patient health records.
PCI DSS	Finance	Ensures secure credit card transactions.
ISO 27001	Global	Framework for managing information security.

Monitoring and Incident Response

- Detect, respond to, and recover from security incidents.

Techniques:

- **Continuous Monitoring**
 - Log analysis and threat detection
- **Security Alerts**
 - Automated alerts for suspicious activities
- **Incident Response Plan**
 - Define response and recovery procedures
- **Forensics & Analysis**
 - Investigate root causes after incidents
- **Examples**
 - AWS CloudWatch, Azure Monitor, Google Cloud Operations

Shared Responsibility Model

- Clearly define security responsibilities between cloud provider and customer.
- **Cloud Provider Responsibilities**
 - Physical data center security
 - Network and infrastructure protection
- **Customer Responsibilities**
 - Data protection, IAM, application security
- **Service Model Differences**
 - *IaaS*: Customer manages OS and apps
 - *PaaS*: Customer manages apps and data
 - *SaaS*: Customer manages user access and data

Cloud Security Mechanisms

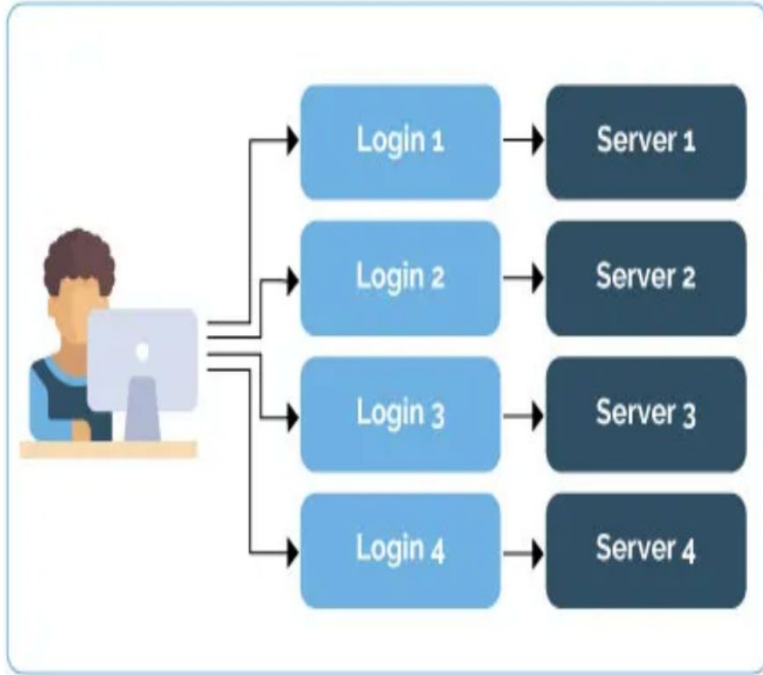
Core Components of Cloud Security Mechanisms:

1. **Authentication & Authorization:**
 - Who are you? What can you do?
2. **Data Integrity & Confidentiality:**
 - Is data altered or leaked?
3. **Accountability:**
 - Can we trace actions?
4. **Cryptography & Access Control:**
 - Foundations for secure operations

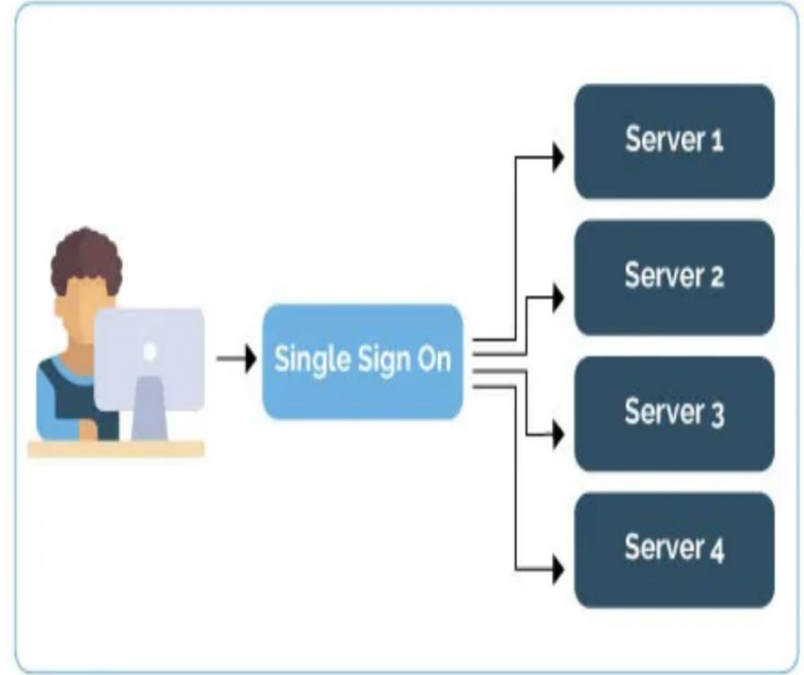
Single Sign-On (SSO)

- Single Sign-On (SSO) allows users to access multiple cloud applications with one set of login credentials.
- SSO is built on a concept called federated identity.
- It enables sharing of identity information across trusted but independent systems.
-
- **How it Works:**
 - User authenticates once via Identity Provider (IdP).
 - Token (like SAML or JWT) is shared across services.
 - Eliminates multiple logins, reducing attack surfaces.
- **Benefits**
 - Centralized control
 - Reduced password fatigue
 - Improved user experience

Without Single Sign On (SSO)



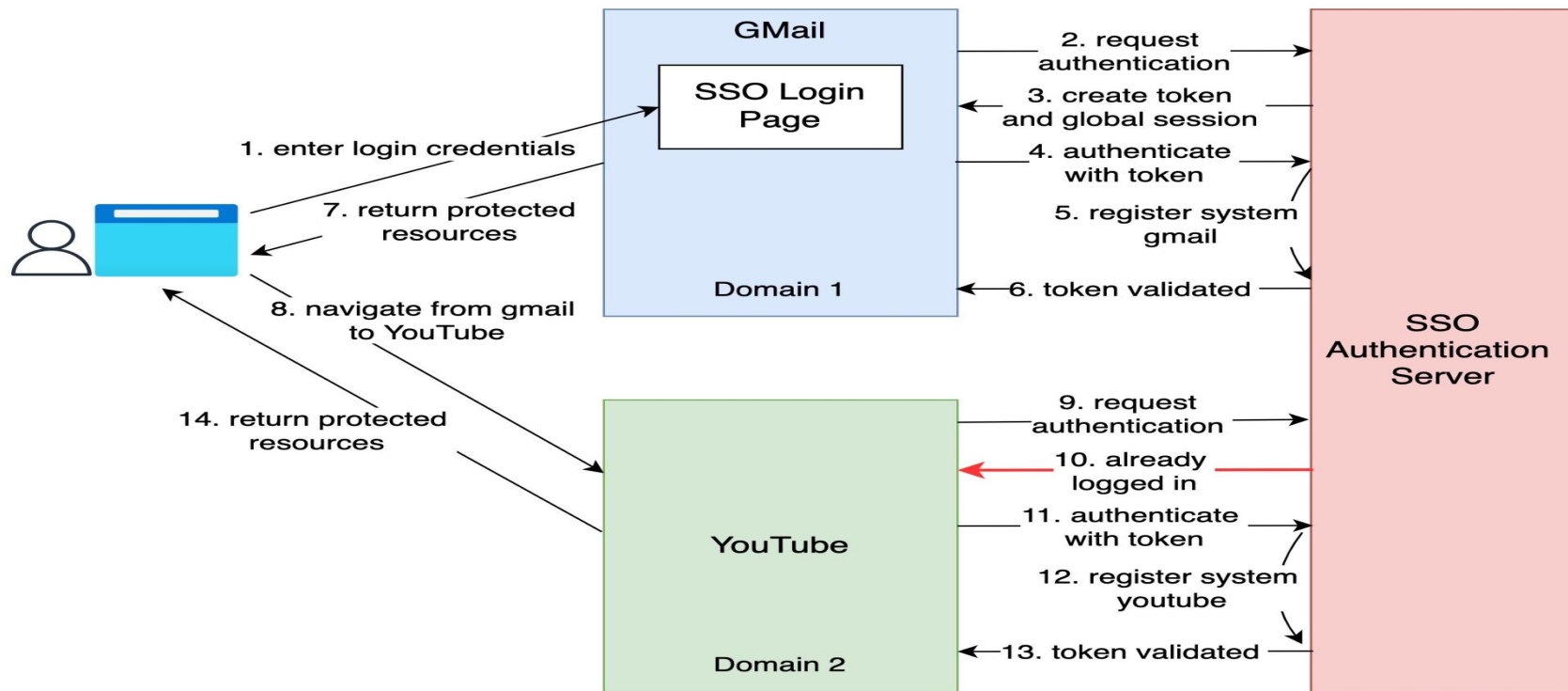
With SSO



SSO – Architecture

- **Protocols Used for authentication process**
 - **SAML (Security Assertion Markup Language)**
 - XML-based open standard for exchanging identity information between services,
 - Enterprise-grade.
 - The SAML assertion is a cryptographically signed XML document that contains information about the user and what the user can access with the service provider.
 - **OAuth 2.0** – Authorization framework for apps (e.g., Google login).
 - **OpenID Connect (OIDC)** –
 - Uses JWT or JSON web token to share identity information between services.
 - Authentication built on OAuth 2.0.
- **Example:**
 - User logs in via Google (IdP).
 - Access granted to Gmail, Drive, Calendar via token exchange.

How does SSO Work?



Step 1: Accessing the First Application (Gmail)

- User visits Gmail
- Gmail checks login status → user is not logged in
- Gmail redirects the user to the SSO Authentication Server
- SSO server also finds no active session
- User is redirected to the SSO login page
- User enters login credentials

Steps 2–3: Authentication & Session Creation

- SSO Authentication Server validates credentials
- Creates a global session for the user
- Generates an authentication token

Steps 4–7: Token Validation by Gmail

- Gmail sends the token to the SSO Authentication Server for validation
- SSO server verifies the token
- Registers Gmail as a trusted system
- Returns “valid” response
- Gmail grants access to the protected resource

Step 8: Accessing Another Application (YouTube)

- From Gmail, user navigates to YouTube
- YouTube checks login status → user is not logged in locally

Steps 9–10: Reusing Existing SSO Session

- YouTube requests authentication from the SSO server
- SSO server finds the user is already logged in
- Returns the existing authentication token

Steps 11–14: Token Validation by YouTube

- YouTube sends token to the SSO Authentication Server
- Token is validated
- YouTube is registered as a trusted system
- SSO server returns “valid”
- YouTube provides access to the protected resource

Identity & Access Management (IAM)

- A framework to control user access in cloud environments.
- IAM defines and manages user identities and their access to cloud resources.
- IAM ensures that the right users have the right access.
- Key IAM Components:
 - Authentication – Verifying user identity (Passwords, MFA).
 - Authorization – Defining user permissions (Role-Based Access Control - RBAC).
 - Single Sign-On (SSO) – One login for multiple cloud applications.
 - Multi-Factor Authentication (MFA) – Extra security beyond passwords.
 - **Audit:** Track user activities for compliance.
- IAM Tools & Services:

AWS IAM, Azure Active Directory, Google Cloud IAM
- Example: A company using AWS IAM Policies to allow only database admins to access RDS.

Key IAM Components

- **Users:** Individual identities with credentials.
- **Groups:** Collection of users sharing the same permissions.
- **Roles:** Temporary access permissions for AWS services.
- **Policies:** JSON documents that define permissions.
- **Access Keys:** Credentials for programmatic access.



- Root account created by default, shouldn't be used or shared
- Users are people within your organization, and can be grouped
- Groups only contain users, not other groups
- Users don't have to belong to a group, and user can belong to multiple groups

IAM Permissions and policies

- Users or Groups can be assigned JSON documents called policies
- These policies define the permissions of the users
- In AWS you apply the least privilege principle: don't give more permissions than a user needs.
- **Policy Structure:** JSON-based format with **Effect**, **Action**, **Resource**, and **Condition** fields.
- **Types of Policies:**
 - Identity-based (Attached to users, groups, or roles)
 - Resource-based (Attached to AWS resources)
- **Managed vs. Inline Policies:**
 - AWS-Managed: Predefined by AWS.
 - Customer-Managed: Created by users for custom access control.
 - Inline: Embedded within a single IAM entity.

IAM policies structure:

Consists of

- Version: policy language version, always include "2012-10-17"

- Id: an identifier for the policy (optional)

- Statement: one or more individual statements (required)

- Statements consists of

- Sid: an identifier for the statement (optional)

- Effect: whether the statement allows or denies access

(Allow, Deny)

- Principal: account/user/role to which this policy applied to

- Action: list of actions this policy allows or denies

- Resource: list of resources to which the actions applied to

- Condition: conditions for when this policy is in effect

(optional)

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "ec2:Describe*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "elasticloadbalancing:Describe*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "cloudwatch:ListMetrics",
        "cloudwatch:GetMetricStatistics",
        "cloudwatch:Describe*"
      ],
      "Resource": "*"
    }
  ]
}
```



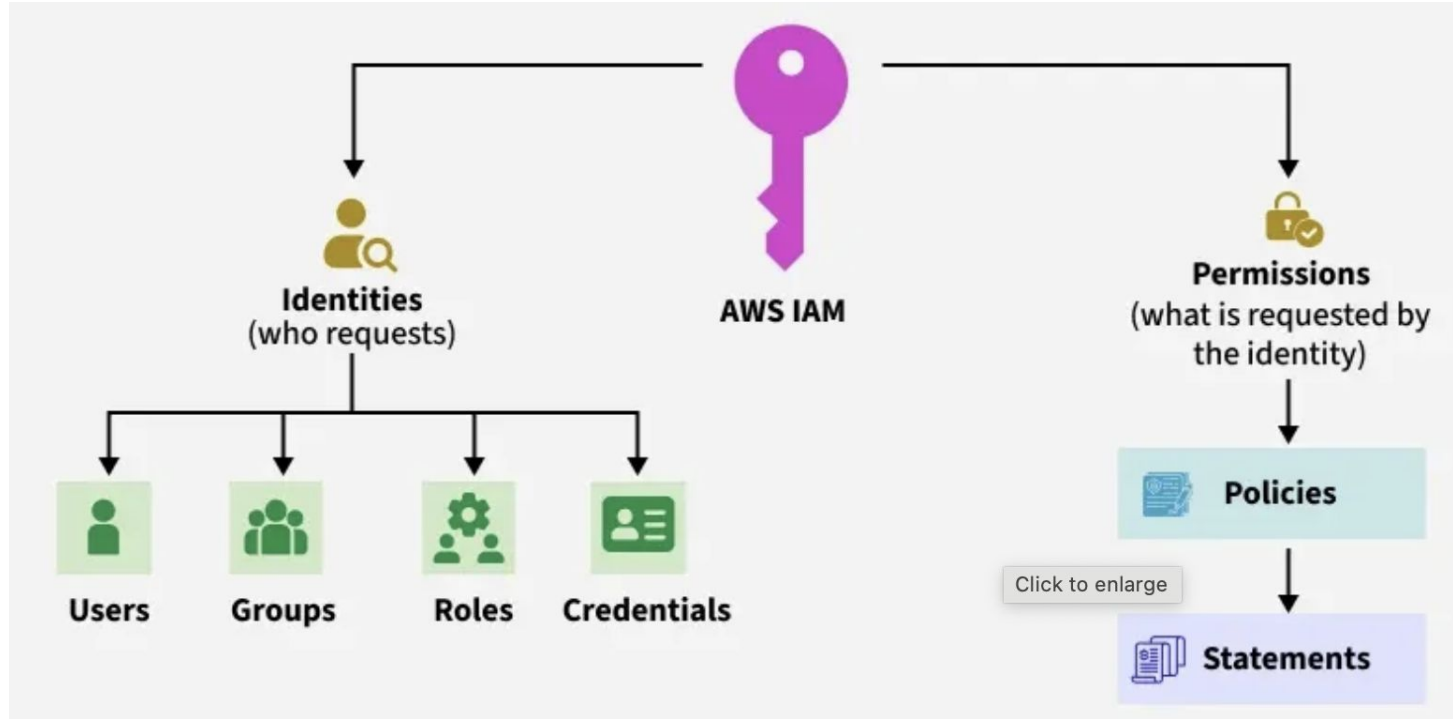
IAM Guidelines & Best Practices

- Don't use the root account except for AWS account setup
- One physical user = One AWS user
- Assign users to groups and assign permissions to groups
- Create a strong password policy
- Use and enforce the use of Multi Factor Authentication (MFA)
- Create and use Roles for giving permissions to AWS services
- Use Access Keys for Programmatic Access (CLI / SDK)
- Audit permissions of your account using IAM Credentials Report & IAM Access Advisor
- Never share IAM users & Access Keys

Multi Factor Authentication - MFA

- Users have access to your account and can possibly change configurations or delete resources in your AWS account
- You want to protect your Root Accounts and IAM users
- MFA = password *you know* + security device *you own*
Password+ MFA => Successful login
- **Main benefit of MFA:**
if a password is stolen or hacked, the account is not compromised

AWS IAM



Hands-on IAM Demo

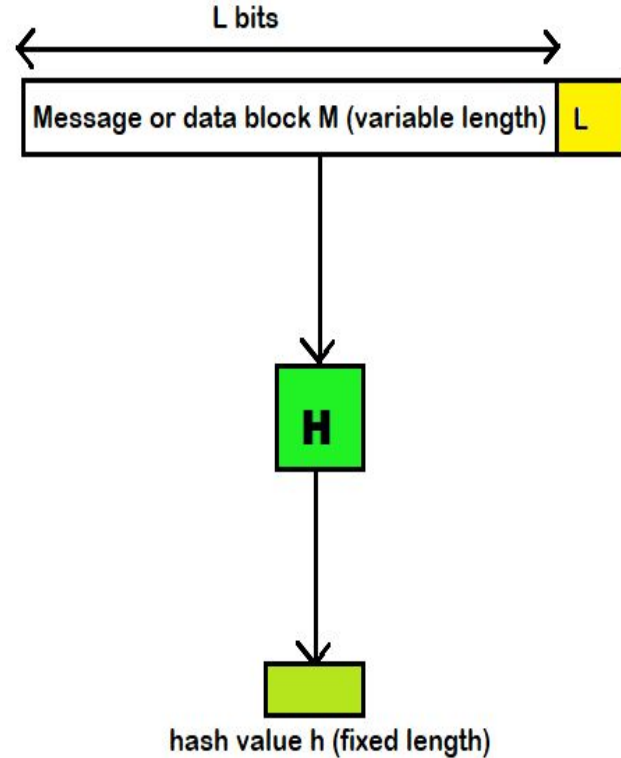
- **Step 1:** Create an IAM User
 - Navigate to IAM Console → Users → Add User
 - Assign appropriate permissions (Administrator/Custom Policy)
- **Step 2:** Create an IAM Group
 - Add multiple users to a group with shared policies
- **Step 3:** Create and Attach a Policy
 - Use AWS Policy Generator or JSON Editor
 - Attach policy to a user, group, or role
- **Step 4:** Implement MFA for IAM User
 - Enable MFA in the Security Credentials section

Hashing

- Hashing is a cryptographic technique that converts input data of any size into a fixed-length value, called a hash value or message digest.
 - The digest is of fixed length
 - It is usually smaller than the original message
- **Key Characteristics**
 - Irreversible: It is a one-way function, cannot reconstruct original
 - Deterministic: Same input always produces the same hash.
 - Avalanche effect: A small change in input produces a completely different hash.
 - It is fast to compute but computationally infeasible to reverse.
- **Common Algorithms:**
 - MD5 (deprecated)
 - SHA-1 (deprecated)
 - SHA-256
 - SHA-512

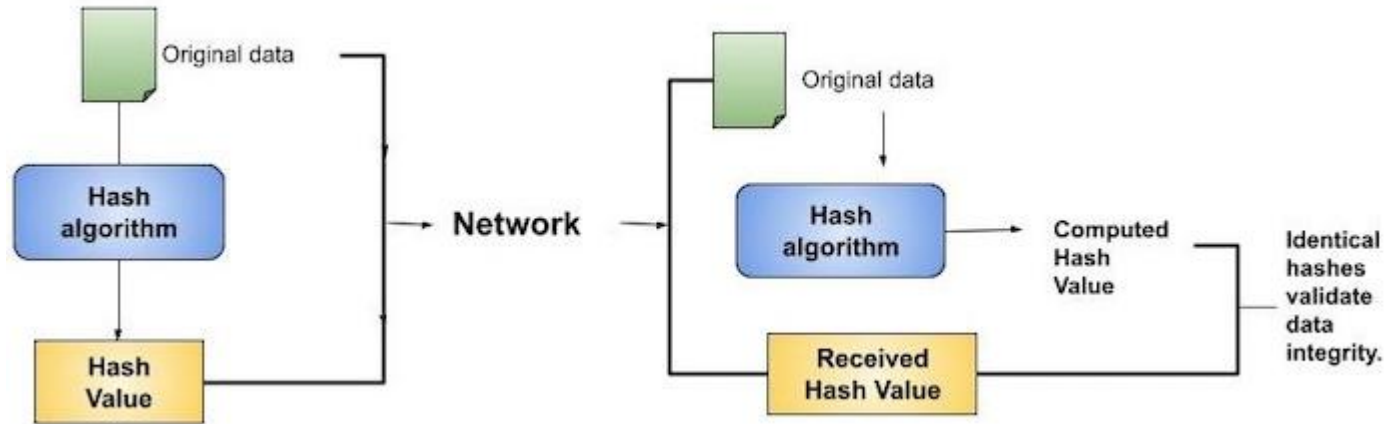
Purpose of Hashing

- Ensures data integrity
- Detects data modification or tampering
- Used for password storage, file verification, and digital signatures



How It Works

1. Input data (message/file) is given to a hash algorithm.
2. The algorithm generates a fixed-size hash value.
3. Receiver recomputes the hash and compares it with the original hash.
4. If both hashes match → data is intact.



Hashing – Use Cases

Applications in Cloud Security:

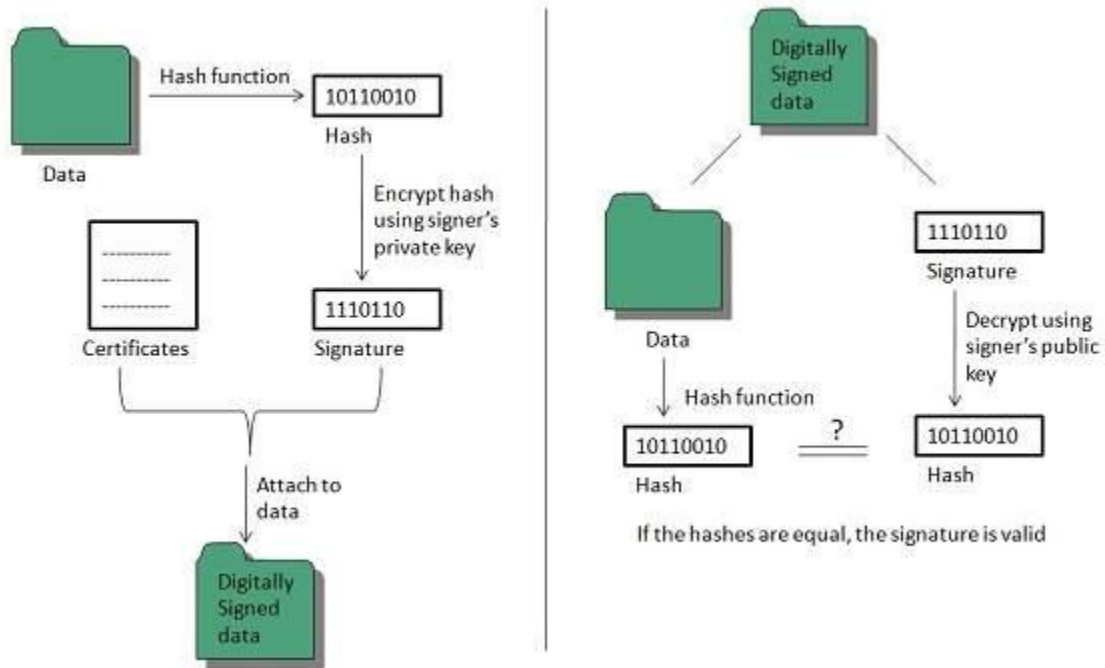
- **Password Storage:** Store hash, not plain text.
- **Integrity Verification**
 - Any change to the original data results in a completely different message digest.
 - This clearly indicates that data tampering has occurred.
- **Blockchain & Auditing:** Immutable record-keeping.
- **Example:**

File Hash before upload $\neq$ Hash after download $\rightarrow$ file tampered.

Digital Signatures

- A digital signature is a cryptographic mechanism used to provide data authenticity and integrity through authentication and non-repudiation.
- A message is assigned a digital signature before transmission.
- If the message undergoes any unauthorized modification after signing, the digital signature becomes invalid.
- A digital signature provides evidence that the message received is exactly the same as the message created and sent by its legitimate sender.

Digital signature working:



Components Used

- Hashing algorithm
- Public-key cryptography
- Private key (signing)
- Public key (verification)

Creation of a Digital Signature

- Both hashing and asymmetric (public-key) encryption are used.
- The sender:
 1. Creates a message digest using a hash function.
 2. Encrypts the message digest using their private key.
 3. Appends the encrypted digest (digital signature) to the original message.

Verification of a Digital Signature

- The recipient:
 1. Uses the sender's public key to decrypt the digital signature.
 2. Obtains the message digest from the signature.
 3. Generates a new digest from the received message.
 4. Compares both digests.
- If the digests match → the signature is valid.

Applications of Digital Signature

- Secure email communication
- Software distribution
- Online transactions
- Cloud authentication services
- Legal and financial documents

Digital Signatures – Use in Cloud

- **Applications:**
 - **SSL/TLS Certificates:**
 - Verify website authenticity.
 - **Software Signing:**
 - Ensure trusted deployments.
 - **Email Encryption (PGP):**
 - Prevent spoofing.
- **Cloud Example:**
 - AWS Signer – automates code signing for Lambda or IoT devices.

Combining IAM + Cryptography

Integrated Protection:

- **IAM** controls who can access resources.
- **Hashing** ensures data integrity.
- **Digital Signatures** validate authenticity.
- **SSO** simplifies secure authentication.

Result:

End-to-end trust and accountability in cloud systems.

Cloud Security Tools & Services

- **AWS Security Services:**
 - AWS IAM – Identity & Access Management
 - AWS Shield – DDoS Protection
 - AWS KMS – Key Management System (encryption)
 - AWS CloudTrail – Logs all API activity
- **Azure Security Services:**
 - Azure AD – Identity & Access
 - Azure Sentinel – Security Information & Event Management (SIEM)
 - Azure DDoS Protection
- **Google Cloud Security Services:**
 - Google IAM – Identity Management
 - Security Command Center – Threat Detection
 - VPC Service Controls – Secure cloud networking

Disaster Management in the Cloud

Disaster:

- Any unexpected event that causes serious disruption to IT systems, data, applications, or business operations.
- It is something that prevents normal services from running and requires special recovery actions.
- In cloud computing, a disaster is not only a natural calamity — it can be technical, human, or cyber-related.

A disaster is an event that:

- Causes system downtime
- Results in data loss or unavailability
- Interrupts business continuity
- Cannot be fixed by normal, routine operations

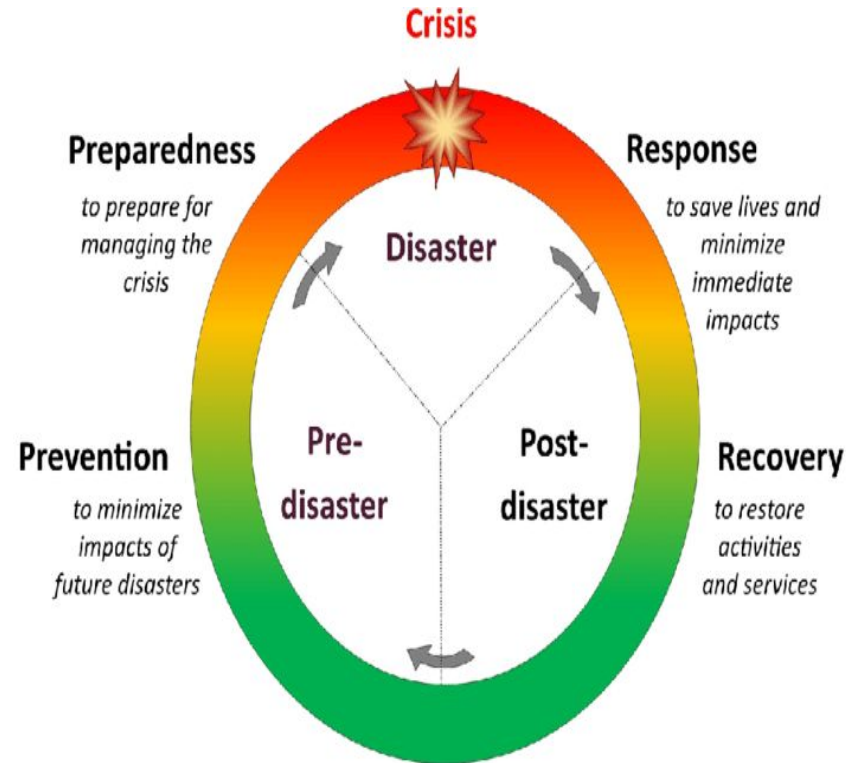
Disaster Management in the Cloud

- Disaster management involves planning, implementing, and maintaining strategies to restore cloud operations during service disruptions.
- **Goal:**
 - Minimize downtime,
 - data loss, and
 - ensure service continuity.
- **Key Aspects:**
 - Data protection (backup, replication)
 - Application availability
 - Infrastructure resilience

Visual: Diagram showing “Users → Cloud Infra → Backup Site”

Cloud Disaster Management Cycle

- The Cloud Disaster Management Cycle is a continuous process that defines how organizations prevent, prepare for, respond to, and recover from cloud-related disasters such as service outages, cyberattacks, data loss, or system failures.
- It is divided into pre-disaster and post-disaster phases.
- Phases:
 - Prevention
 - Preparedness
 - Crisis
 - Response
 - Recovery



1. Prevention (Pre-Disaster)

Purpose: To minimize the impact of future cloud disasters.

- Implement strong cloud security controls (IAM, encryption, firewalls)
- Apply least-privilege access and network segmentation
- Use secure configurations and hardened virtual machines
- Regular patching and vulnerability management
- Compliance with cloud security standards

Goal: Reduce the likelihood and impact of cloud failures

2. Preparedness (Pre-Disaster)

Purpose: To ensure readiness for cloud incidents.

- Develop a Cloud Disaster Recovery (DR) plan
- Define RTO (Recovery Time Objective) and RPO (Recovery Point Objective)
- Configure automated backups and snapshots
- Set up multi-region or multi-zone deployments
- Conduct DR drills and failover testing

Goal: Be ready to manage cloud outages effectively

3. Cloud Disaster / Crisis

This phase represents the actual cloud incident, such as:

- Data breach or ransomware attack
- Cloud service outage
- Accidental data deletion
- Network or region failure
- Application crash due to misconfiguration

4. Response (Post-Disaster)

Purpose: To minimize immediate damage and restore critical services.

- Activate the incident response plan
- Isolate affected cloud resources
- Perform failover to backup systems or regions
- Notify stakeholders and users
- Begin forensic analysis and damage assessment

Goal: Ensure service continuity and reduce downtime.

5. Recovery (Post-Disaster)

Purpose: To restore normal cloud operations and improve resilience.

- Restore data from backups
- Rebuild compromised cloud resources
- Validate data integrity and system performance
- Resume normal operations
- Update DR and security policies based on lessons learned

Goal: Return to stable and secure cloud operations

Data Backup, Redundancy & Replication

- **Data Backup:**
 - Periodic copies stored in different regions or storage tiers.
- **Redundancy:**
 - Duplicate systems (hardware/software) prevent single-point failure.
- **Replication:**
 - Real-time data copy across availability zones or regions.
 - Synchronous = real-time; zero RPO
 - Asynchronous = delayed; better for cost savings
- **Example:**
 - AWS S3 Cross-Region Replication,
 - Azure Geo-Redundant Storage

Disaster Recovery

- Strategy to backup critical applications, systems, and data to the cloud
- Enables quick and reliable restoration after a disaster
- Uses automated backups to reduce manual effort
- Provides scalable cloud storage based on need
- Ensures easy and fast access to data after outages
- Protects against:
 - Power failures
 - Natural disasters
 - Hardware failures
 - Cyberattacks
- More cost-effective than traditional on-premises DR
- Easier to deploy and manage
- Highly scalable and flexible

Why disaster recovery?

- Protects data:
 - Minimizes data loss and ensures quick recovery.
- Ensures business continuity:
 - Allows operations to resume quickly after a disruption.
- Reduces financial impact:
 - Minimizes revenue loss, fines, and recovery costs.
- Maintains reputation:
 - Protects brand image and customer trust.
- Meets compliance requirements:
 - Helps adhere to data privacy laws and industry standards.

Types of disaster recovery

1. Data Center Disaster Recovery
2. Network Disaster Recovery
3. Virtualized Disaster Recovery
4. Cloud Disaster Recovery
5. Disaster Recovery as a Service (DRaaS)

1. Data Center Disaster Recovery

- Focuses on restoring full IT infrastructure (servers, storage, racks, physical facilities) using backups and failover sites, often in secondary data centers or colocation facilities.

2. Network Disaster Recovery

- Ensures that network services — including connectivity, routing, and access to backup systems — can be restored quickly after an outage.

3. Virtualized Disaster Recovery

- Uses virtualization technology to replicate and run systems (as VMs) in alternate locations, speeding up recovery and reducing hardware dependency.

4. Cloud Disaster Recovery

- Instead of relying on physical infrastructure, systems and data are backed up and replicated to cloud platforms.
- Cloud recovery can offer scalable, cost-efficient failover and restoration.

5. Disaster Recovery as a Service (DRaaS)

- A managed cloud-based DR offering where a third-party provider hosts and orchestrates replication, failover, and recovery under agreed service levels.

Disaster Recovery Strategies

Strategy	Description	Recovery Time
Backup & Restore	Simple periodic backups	Hours–Days
Pilot Light	Minimal infrastructure running	Minutes–Hours
Warm Standby	Partially running environment	Minutes
Multi-site Active–Active	Fully running replicas	Near zero

Business Continuity Planning (BCP)

- **Objective:**
 - Maintain essential business operations during disruption.
- **Components:**
 - Risk Assessment
 - Critical Process Identification
 - Disaster Recovery Plan Integration
 - Employee Communication Protocols

RTO & RPO

- **RTO (Recovery Time Objective):**
 - Maximum acceptable downtime before operations resume.
- **RPO (Recovery Point Objective):**
 - Maximum acceptable data loss measured in time.
- **Example:**
 - RTO = 1 hour → must restore service in 1 hour.
 - RPO = 15 min → can lose only 15 minutes of data.

Cloud Disaster Recovery Tools

Cloud Provider	Tool/Service	Purpose
AWS	AWS Backup, CloudEndure DR	Backup, cross-region recovery
Azure	Azure Site Recovery	VM replication & failover
Google Cloud	Backup & DR Service	Centralized backup management
IBM Cloud	Cloud Backup	Managed DR orchestration

Emerging Trends in Cloud Security

- **Zero Trust Architecture:**
 - Never trust, always verify.
- **Confidential Computing:**
 - Data encrypted even during processing (in-use encryption).
- **AI-driven Threat Detection:**
 - Machine learning for anomaly detection & predictive alerts.
- **Automated Security Orchestration:**
 - Policy-based auto-remediation.

Security Challenges in Cloud Computing

- **Data Breaches & Unauthorized Access**
 - Compromise of sensitive data through weak credentials, stolen keys, or malicious attacks.
- **Misconfiguration & API Vulnerabilities**
 - Improper cloud setup or insecure APIs exposing services to attackers.
- **Insider Threats**
 - Employees or partners misusing access privileges or leaking confidential data.
- **Data Loss & Leakage**
 - Accidental deletion, poor backup, or insecure data transfer leading to loss or exposure.

- **Compliance & Legal Risks**
 - Failure to meet regional data protection laws like GDPR, HIPAA, or ISO 27001.
- **Multi-Cloud Policy & Key Management Issues**
 - Difficulty maintaining consistent access control, encryption, and monitoring across multiple providers.
- **Limited Visibility & Monitoring Gaps**
 - Insufficient logging and oversight delay detection and response to incidents.
- **Denial of Service (DoS) Attacks**
 - Overwhelming cloud resources to disrupt availability and business operations.

Thank you